

Algoritmi e strutture dati per i Big Data

Memorizzazione e pre-elaborazione
Prof.ssa Paola Vocca



Contents

- ☐ Big data storage
- ☐ Structured and unstructured data storage
- ☐ NoSQL DataBase
- ☐ NoSQL Data Models
- ☐ Introduction to MongoDB
- ☐ Installing MongoDB
- ☐ CRUD Operations in MongoDB
- ☐ MongoDB Indexing
- ☐ MongoDB Replication
- ☐ MongoDB Sharding
- ☐ MongoDB Security
- ☐ MongoDB Query

Big Data Storage

❑ Relational DBMSs (RDBMSs)

- Traditional technology to store structured data in web and business applications

❑ SQL is good

- Rich language and toolset
- Easy to use and integrate
- Many vendors

❑ RDBMSs promise ACID guarantees

ACID properties

❑ Atomicity

- All statements in a transaction are either executed or the whole transaction is aborted without affecting the database: “all or nothing” rule that is, transactions do not occur partially

❑ Consistency

- A database is in a consistent state before and after a transaction; it refers to the correctness of a database

❑ Isolation

- Transactions cannot see uncommitted changes in the database (i.e., the results of incomplete transactions are not visible to other transactions)

❑ Durability

- Changes are written to disk (i.e., non-volatile memory) before a database commits a transaction so that committed data cannot be lost if a system failure occurs

Pros and cons of RDBMS

Pros

- ✓ Well-defined consistency model
- ✓ ACID guarantees
- ✓ Relational integrity maintained through entity and referential integrity constraints
- ✓ Well suited for OLTP apps
- ✓ Sound theoretical foundation
- ✓ Stable and standardized DBMSs available
- ✓ Well understood

Cons

- ✗ Performance as major constraint, scaling is difficult
- ✗ Limited support for complex data structures
- ✗ Complete knowledge of DB schema required to build new queries
- ✗ Commercial DBMSs are expensive
- ✗ Some DBMSs have field size limits
- ✗ Data integration from multiple RDBMSs can be cumbersome

❑ Workload spikes

- Internet-scale data size
- High read-write rates
- Frequent schema changes

❑ Let's scale RDBMSs

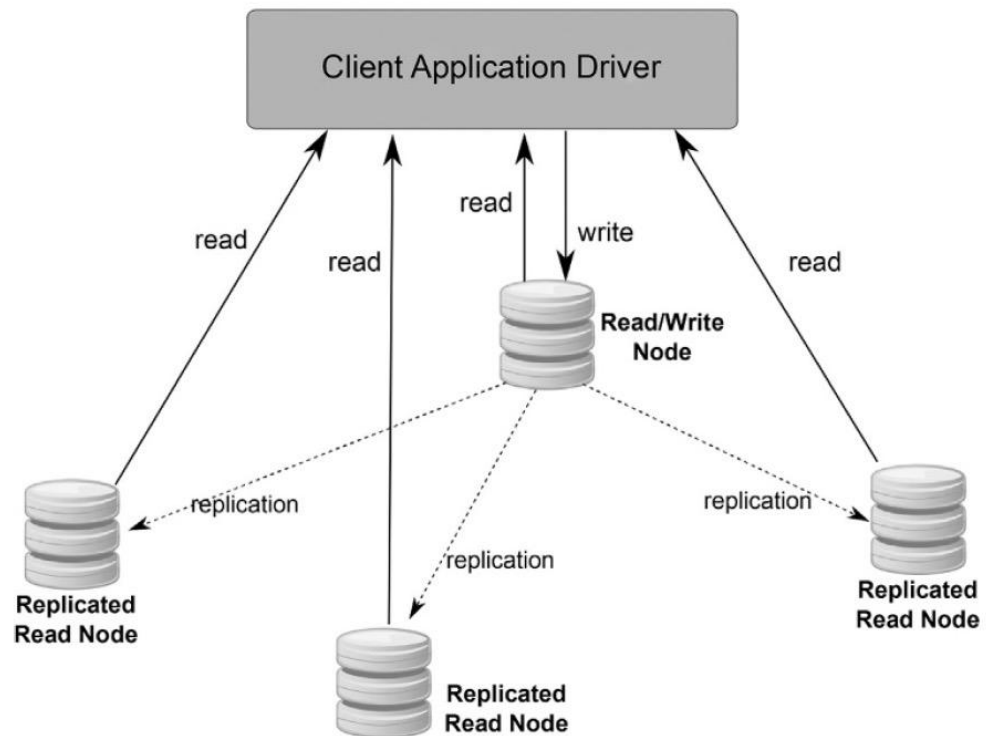
- But they were not designed to be distributed

❑ How to scale RDBMSs?

- Replication
- Sharding

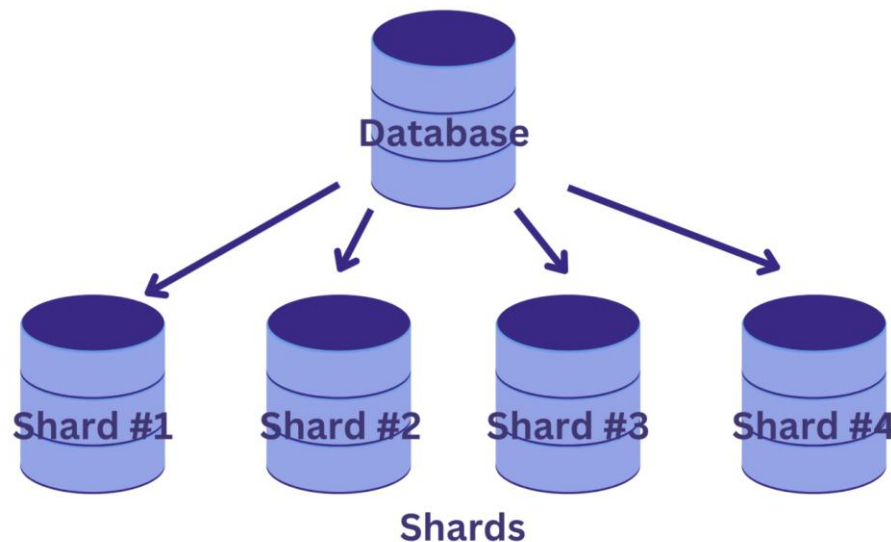
Replication

- ❑ Primary backup with master/worker architecture
- ✓ Replication improves read scalability
- ✗ Write scalability?

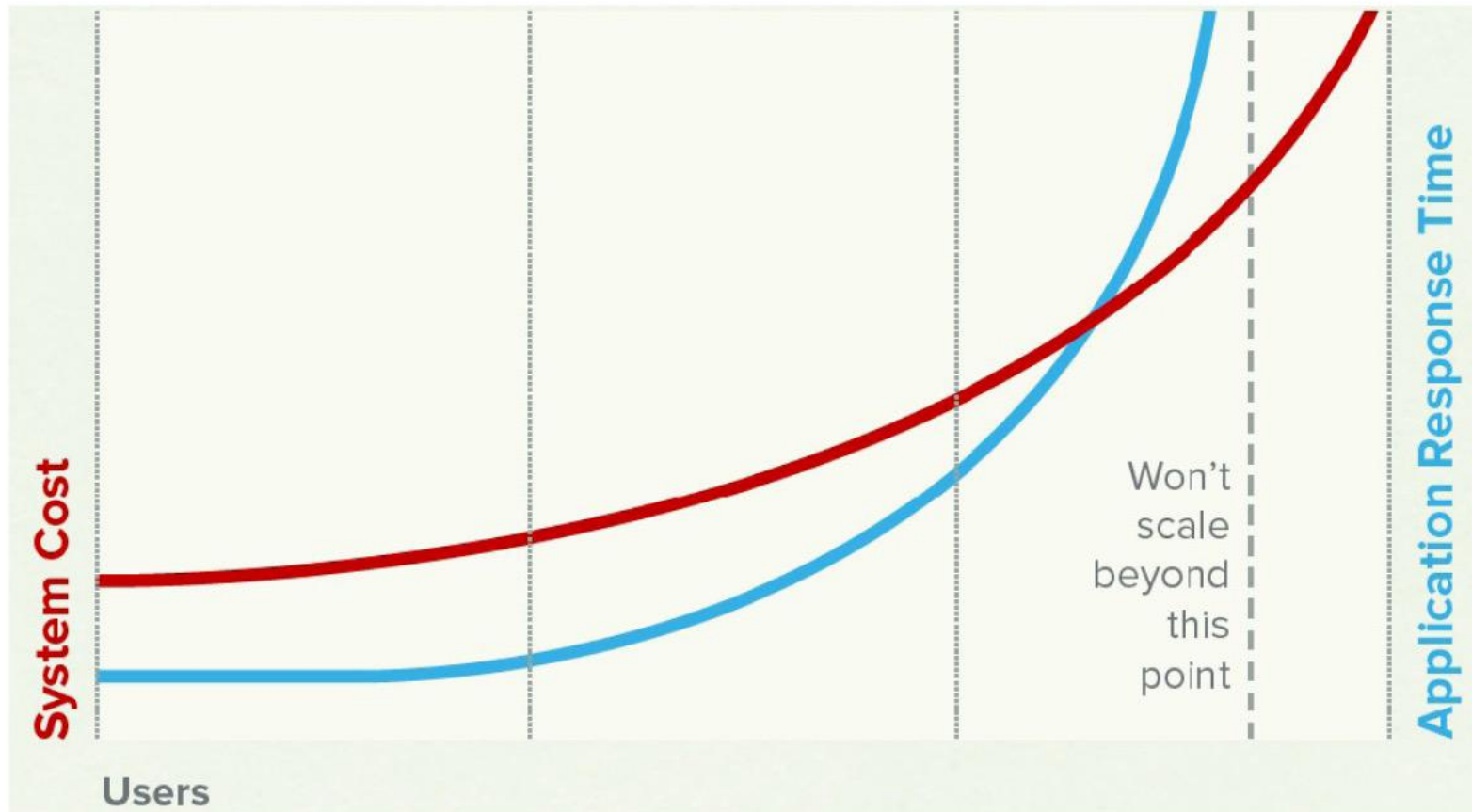


Sharding

- ❑ Horizontal partitioning of data across servers
- ✓ Read and write operations scale
- ✗ Cannot execute transactions across shards (partitions)
- ❑ Consistent hashing can be used to determine which server any shard is assigned to
 - Hash both data and server using the same hash function in the same ID space



Scaling RDBMSs is expensive and inefficient



Source: Couchbase technical report

NOSQL DB

❑ **Definition:** NoSQL databases are designed to handle unstructured or semi-structured data at scale, providing flexibility for modern applications.

❑ **Why NoSQL?**

- Scalability
- Flexibility with data models
- High availability

□ NoSQL = **Not Only SQL**

- SQL-style querying is not the crucial objective



Requirements and features

- ❑ Large volumes of data.....increasing
- ❑ No regular data structure to manage
- ❑ Relatively homogeneous elements among them
(no relationship between them)
- ❑ Simple types of operation

Needs and characteristics

- ❑ Example: **Twitter (set of users who publish tweets)**
- ❑ Few collections of interest (two entities: users and tweets), but massive
- ❑ Few operations (insert/update user, insert tweet)
- ❑ Data identified by a key, but only partially structured

Needs and characteristics

- ❑ **“One size does not fit all”**
- ❑ Great scalability (many processors, horizontal data partitioning, distributed architecture at low cost)
- ❑ High availability, Replication and Eventual Consistency
- ❑ High Performance Data Access

❑ “One size does not fit all”

❑ Replication

- Master-Slave Replication
- Master-Master Replication

❑ Scalability

- Sharding Files
- High performance to Data Access

“One size does not fit all”

- ❑ Relational model as a base, but it's not enough
- ❑ Not requiring a schema
- ❑ Adaptability to different application scenarios
- ❑ Languages for semistructured data : JSON, XML
- ❑ Less powerful Query languages (CRUD or SCRUD operations)

NoSQL data stores: main features

- ❑ Support **flexible schema**
 - No requirement for fixed rows in a table schema
 - Well suited for agile development process
- ❑ Support **horizontal scaling**
 - Partitioning of data and processing over multiple nodes
- ❑ Provide **high availability**
 - By replicating data in multiple nodes, often geo-distributed
- ❑ Mainly utilize shared-nothing architecture
 - **With exception of graph-based databases**
- ❑ Avoid **unneeded complexity**
 - E.g., elimination of join operations
- ❑ Support **weaker consistency models**
 - BASE rather than ACID: compromising reliability for better performance

CAP Theorem

❑ In a distributed system is not possible to guarantee simultaneously: **C**onsistency, **A**vailability, **P**artition tolerance

Alternatively,

❑ In a distributed system **Pick two** of **C**onsistency, **A**vailability, **P**artition tolerance

ACID vs BASE: ACID

- ❑ Two design philosophies at opposite ends of the **consistency-availability** spectrum
 - **CAP theorem**
- ❑ **ACID**: traditional approach for RDBMSs
 - **High Precautionary approach**: prevents conflicts from occurring
- ❑ Usually implemented with write locks managed by system
- ❑ Leads to performance degradation and deadlocks (hard to prevent and debug)
 - Does not scale well when handling petabytes of data (remember of latency!)

□ **BASE**: Basically **A**vailable, **S**oft state, **E**ventual consistency

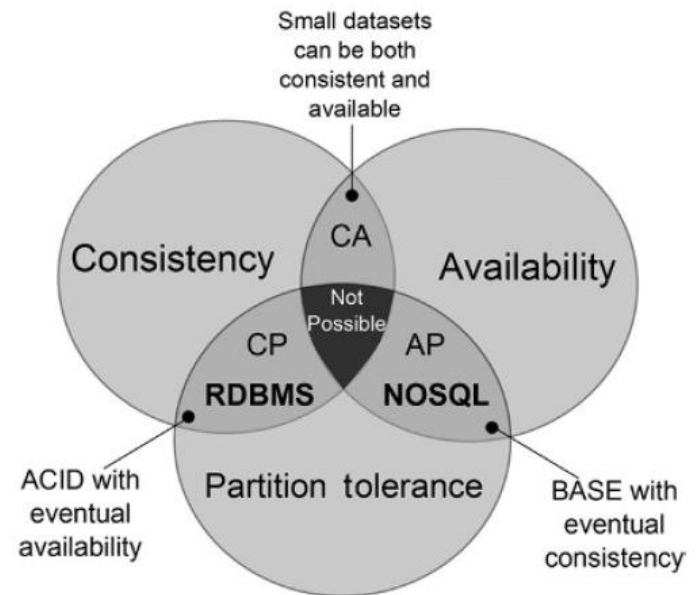
- **Basically Available**: the system is available most of the time and there could exist a subsystem temporarily unavailable
- **Soft state**: data is not durable that is, its persistence is in the hand of the user that must take care of refreshing it
- **Eventually consistent**: the system eventually converges to a consistent state

□ **Optimistic** approach

- Let's conflicts occur, but detects them and takes action to sort them out: how?
 - **Conditional updates**: test the value just before updating
 - **Save both updates**: record that they are in conflict and then merge them

NoSQL and consistency

- ❑ Biggest change from RDBMS
 - RDBMS: strong consistency
 - Traditional RDBMS are either CA systems or CP systems (depending on configuration)
- ❑ Majority of NoSQL systems provide eventual consistency (i.e., AP systems)
- ❑ Some NoSQL systems provide strong consistency or tunable consistency
 - E.g., Cassandra and MongoDB



Pros and cons of NoSQL

- ✓ Easy to scale-out
- ✓ Higher performance for massive data scale
- ✓ Allow data sharing across multiple servers
- ✓ High availability and fault tolerance provided by data replication
- ✓ Most are either open-source or cheaper
- ✓ Support complex data structures and objects
- ✓ No fixed schema, support unstructured data
- ✓ Fast retrieval of data, suitable for real-time apps
- ✗ Many do not support ACID, less suitable for OLTP apps
- ✗ No common data storage model -> no well-defined approach for design
- ✗ Lack of standardization (e.g., standard query language)
- ✗ Many do not support join ops
- ✗ Lack of reference model can lead to solution lock-in

NoSQL Data Models

- ❑ **Data model:** set of constructs for representing information
 - Relational model: tables, columns and rows
- ❑ **Storage model:** how the data store management system stores and manipulates data internally
- ❑ A data model is usually independent of the storage model.

our Main Types of NoSQL Databases

□ Each category is based on a specific data organization

1. **Key-Value Store**
2. **Document Store**
3. **Column-Family Store**
4. **Graph Database**

❑ **Description:** A simple model where data is stored as **key-value pairs**.

- Like a dictionary where each key is unique, and the value can be any type of data.

❑ **Use Cases:**

- Caching (e.g., Redis)
- Session management
- Simple data storage

❑ **Example Databases:**

- Redis
- Amazon DynamoDB

□ Description:

- **Stores data as documents**, typically JSON, BSON, or XML, where each document contains data in key-value format but with a complex, hierarchical structure.
- Like key-value stores but **allows querying of document content**.

□ Use Cases:

- Content management systems
- E-commerce catalogs
- Mobile app data storage

□ Example Databases:

- MongoDB
- Couchbase

□ Description:

- Stores data in rows and columns, but each row doesn't have to have the same columns.
- Optimized for read/write of large volumes of data across many servers.

□ Use Cases:

- Time-series data
- Large-scale data warehousing
- Real-time analytics

□ Example Databases:

- Apache Cassandra
- HBase

□ Description:

- Uses nodes, edges, and properties to represent and store data, where relationships between entities (edges) are as important as the entities themselves (nodes).
- Focuses on how data points relate to one another.

□ Use Cases:

- Social networks Recommendation engines Fraud detection

□ Example Databases:

- Neo4j
- Amazon Neptune

Key Differences Between NoSQL Databases

□ Data Model

- **Key-Value:** Simple key-value pairs
- **Document:** Semi-structured, JSON-like documents
- **Column-Family:** Wide-column data, schema-agnostic
- **Graph:** Nodes and edges representing relationships

□ Query Flexibility

- **Key-Value:** Limited query capability
- **Document:** Rich querying on documents
- **Column-Family:** Efficient reads/writes on wide-column data
- **Graph:** Traversing relationships between nodes

□ Scalability

- **Key-Value, Document, and Column-Family** databases are **horizontally scalable** (can grow by adding more servers)
- **Graph** databases can **scale vertically or horizontally** but are often more difficult to distribute.

When to Use Which Type

- ❑ **Key-Value Store:** When simplicity and performance are the primary concerns.
- ❑ **Document Store:** When working with semi-structured data and need flexible schemas.
- ❑ **Column-Family Store:** When dealing with high write/read volumes and large-scale distributed data.
- ❑ **Graph Database:** When relationships between data points are crucial, such as in social networks or recommendations.

Introduction to MongoDB

What is MongoDB?

❑ **Definition:** MongoDB is a NoSQL, document-oriented database used for high-volume data storage.

❑ **Key Features:**

- Schema-less, flexible data model.
- JSON-like documents.
- Scalable, distributed database architecture.

Why MongoDB?

Benefits:

- ❑ Flexible schema design.
- ❑ Horizontal scaling for large datasets.
- ❑ High performance for read/write operations.
- ❑ Native replication and sharding for fault tolerance and scalability.

Key Concepts of MongoDB

- ❑ **Document**: BSON format (Binary JSON) containing key-value pairs.
- ❑ **Collection**: Group of documents, similar to a table in RDBMS.
- ❑ **Database**: Stores collections.
- ❑ **Shard**: A partition of data for distributed storage.
- ❑ **Replica Set**: Group of MongoDB servers that maintain the same data for redundancy and high availability.

MongoDB vs Traditional RDBMS

MongoDB	Relational DB (e.g., MySQL)
Document-oriented	Table-based
No fixed schema	Fixed schema
Supports nested structures	Flat tables
Horizontal scaling	Vertical scaling

Use Cases for MongoDB

- ❑ Content management systems (CMS)
- ❑ E-commerce product catalogs
- ❑ Real-time analytics
- ❑ IoT applications
- ❑ Social networks

Installing MongoDB

Installing MongoDB (Windows)

1. Download MongoDB from the official site:
<https://www.mongodb.com/try/download/community>
2. Run the installer and follow the prompts.
3. Choose "**Complete**" installation.
4. Add MongoDB to the system **PATH** for ease of use.
5. Install MongoDB as a service (optional).

Installing MongoDB (Mac)

1. Open the terminal.
2. Install Homebrew if it's not installed:

```
$ /bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

3. Run the command:

```
$ brew tap mongodb/brew
```

```
$ brew install mongodb-community@5.0
```

Installing MongoDB (Linux-Ubuntu)

1. Import the public key used by the package management system:

```
$ wget -qO - https://www.mongodb.org/static/pgp/server-5.0.asc | sudo apt-key add -
```

2. Create a list file for MongoDB:

```
$ echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu focal/mongodb-org/5.0 multiverse" | sudo tee /etc/apt/sources.list.d/mongodb-org-5.0.list
```

3. Install MongoDB:

```
$ sudo apt-get install -y mongodb-org
```

❑ Connecting to MongoDB:

- Run the MongoDB shell using the command:

```
$ mongo
```

- You will now be connected to the `test` database.

❑ Basic Commands:

- View Databases:

```
show dbs
```

- Use a Database:

```
use <db_name>
```

- View Collections:

```
show collections
```

CRUD Operations in MongoDB

CRUD Operations in MongoDB

❑ Create:

- Insert a new document into a collection:

```
db.collection.insertOne({name: "John", age: 25})
```

❑ Read:

- Find documents in a collection:

```
db.collection.find()
```

❑ Update:

- Update existing documents:

```
db.collection.updateOne({name: "John"}, {$set: {age: 26}})
```

❑ Delete:

- Remove documents:

```
db.collection.deleteOne({name: "John"})
```

- ❑ **Flexible Schema:** MongoDB allows dynamic, schema-less designs.
- ❑ **Nested Documents:** Example:

```
{  
  name: "John",  
  contact: {  
    email: "john@example.com",  
    phone: "123-456-7890"  
  }  
}
```

MongoDB Indexing

❑ What is Indexing?

- Improves the speed of data retrieval.

❑ Create an Index:

- Example:

```
db.collection.createIndex({name: 1})
```

❑ Types of Indexes:

- Single Field Index
- Compound Index
- Multikey Index

MongoDB Indexing: Single Field Index

❑ Description:

- A single field index is the simplest type of index in MongoDB. It is created on a single field of a collection. This type of index enhances the query performance when querying based on that single field.

❑ **Example:** If you have a collection `users` with a field `age`, you can create a single field index on age:

```
db.users.createIndex({ age: 1 }) // ascending index
```

❑ When to Use:

- Use a single field index when you frequently query or sort based on that single field.
- It improves query efficiency, especially for equality and range queries.

MongoDB Indexing: Compound Index

❑ Description:

- A **compound index** is an index created on multiple fields of a document. It allows MongoDB to efficiently support queries that specify conditions on multiple fields.

❑ **Example:** If you frequently query on both the `name` and `age` fields, you can create a compound index:

```
db.users.createIndex({ name: 1, age: -1 }) // ascending on `name` and descending on `age`
```

❑ When to Use:

- Compound indexes are used when you often perform queries that involve multiple fields.
- They also support queries on the prefix fields. For instance, if you create a compound index on `{name, age}`, you can still use it to optimize queries that only include `name`.

MongoDB Indexing: Multikey Index

❑ Description:

- A **multikey index** is created on fields that contain arrays. MongoDB automatically creates a multikey index if an indexed field contains an array.
- MongoDB will index each element of the array separately.

❑ **Example:** If you have a **tags** field that stores an array of tags, MongoDB will create a multikey index as follows:

```
db.posts.createIndex({ tags: 1 })
```

❑ The index will be created on each value of the **tags** array.

❑ When to Use:

- Use a multikey index when you store arrays in fields and need to search or filter based on the individual elements of those arrays.
- Multikey indexes are ideal for queries that involve matching specific elements within an array.

MongoDB Replication

□ What is Replication?

- **Replication** is the process of synchronizing data across multiple servers.
- In MongoDB, replication provides **data redundancy** and **increased data availability**

□ Replica Set is a group of MongoDB servers that maintain the same data. Each set typically has:

- **1 Primary Node** (handles all write operations)
- **Multiple Secondary Nodes** (replicate the primary's data)

□ Why Use Replication?

- **High Availability:** If the primary node fails, a secondary can automatically take over.
- **Fault Tolerance:** Data is copied to multiple servers, reducing the risk of data loss.
- **Scalability:** Read operations can be distributed across secondaries, improving performance.

How MongoDB Replication Works

❑ Primary Node:

- Handles all **write** operations.
- Replicates data to secondary nodes.

❑ Secondary Nodes:

- Receive data from the primary node via the **oplog** (operation log).
- Can be used for **read** operations in a **read-heavy** workload (read preference).

❑ Automatic Failover:

- If the **primary** goes down, an **election** is held, and a **secondary** becomes the new primary.

❑ Arbiter:

- Participates in the election process but does not store data.
- Used to maintain a quorum and ensure elections can proceed.

❑ Example Replica Set Configuration:

- 1 Primary
- 2 Secondaries
- 1 Arbiter

How MongoDB Replication Works

❑ Initiating a Replica Set

```
rs.initiate()
```

❑ Adding a Secondary Node:

```
rs.add("secondary_host:port")
```


MongoDB Sharding

❑ What is Sharding?

- A method for distributing data across multiple servers.

❑ When to Use Sharding?

- When your dataset is too large for a single server.

❑ **Shard Key:** A field that determines how data is distributed.

MongoDB Sharding Overview

❑ What is MongoDB Sharding?

- **Sharding** is a method for distributing data across multiple machines.
- It is used to **horizontally scale** a MongoDB database, enabling it to handle large datasets and high-throughput operations.

❑ Key Concepts:

- **Shard**: A single MongoDB instance that holds a subset of the database data.
- **Sharded Cluster**: A group of shards that collectively store the entire dataset.
- **Config Servers**: Store metadata and configuration settings for the cluster.
- **Query Router (Mongos)**: Routes queries to the appropriate shards.

❑ Why Use Sharding?

- **Scalability**: As data grows, more shards can be added to scale horizontally.
- **Performance**: Distributes read and write operations across multiple servers, improving performance for large-scale applications.
- **Capacity**: Sharding enables the database to handle datasets that exceed the capacity of a single machine.

How MongoDB Sharding Works

❑ Sharding Key:

- The **sharding key** determines how data is distributed across shards.
- MongoDB divides data into chunks based on the sharding key, which are distributed across shards.

❑ Key Sharding Types:

- **Range-Based Sharding:**
 - Data is divided based on a range of values for the sharding key.
 - Example: `{ "order_id": 1 } to { "order_id": 1000 }`
- **Hash-Based Sharding:**
 - Data is distributed by hashing the sharding key, ensuring even data distribution.
 - Example: Hash of `user_id`

❑ Components of a Sharded Cluster:

1. **Shards:** Store the actual data.
2. **Mongos Router:** Directs client requests to the appropriate shard.
3. **Config Servers:** Store the cluster's metadata and track the location of chunks.

MongoDB Sharding: Example Commands

❑ Enabling Sharding

```
sh.enableSharding("myDatabase")
```

❑ Sharding a Collection:

```
sh.shardCollection("myDatabase.myCollection", { "shardKeyField": 1 })
```

MongoDB Security

□ Authentication:

- Enable authentication for secure access.
- Example: Use `mongod --auth` to enable authentication.

□ Role-Based Access Control (RBAC):

- Assign roles to users:

```
db.createUser({user: "admin", pwd: "password",  
roles: ["root"]})
```

MongoDB Query

Querying in MongoDB

- ❑ MongoDB uses a flexible, JSON-like query language.
- ❑ Key methods for querying:
 - `find()` : Retrieves documents.
 - `findOne()` : Retrieves a single document.
- ❑ Queries are structured using key-value pairs.

Basic Query Example

□ Syntax:

- `db.collection.find({<field>: <value>})`

□ Example:

Find all documents where the "name" is "John":

```
db.students.find({name: "John"})
```

Query with Multiple Conditions

- ❑ You can query using multiple conditions by passing multiple fields in the query object.

- ❑ Example:

Retrieves all documents where the "name" is "John" and "age" is 25.

```
db.students.find({name: "John", age: 25})
```

Using Comparison Operators

❑ MongoDB provides several comparison operators to create more complex queries:

- `$gt`: Greater than
- `$gte`: Greater than or equal to
- `$lt`: Less than
- `$lte`: Less than or equal to
- `$ne`: Not equal

❑ Example: Find all students older than 20:

```
db.students.find({age: {$gt: 20}})
```

Query with Logical Operators

❑ MongoDB supports logical operators like:

- `$and`: Match all conditions.
- `$or`: Match any condition.
- `$not`: Negates a condition.

❑ **Example**: Find all students whose name is "John" or age is greater than 20:

Projection in Queries

❑ **Projection** allows you to specify which fields should be returned in the query results.

❑ **Syntax:**

```
- db.collection.find(<query>, {<field1>: 1, <field2>: 1})
```

❑ **Example:** Retrieve only the name and age fields for all documents where the age is greater than 20:

```
db.students.find({age: {$gt: 20}}, {name: 1, age: 1})
```

Querying Arrays

- ❑ MongoDB supports querying fields that store arrays.
- ❑ **Example:** Find documents where the "subjects" field contains "Math"

```
db.students.find({subjects: "Math"})
```

- ❑ **Example** (with `$all` operator):
 - Find documents where "subjects" contains both "Math" and "Science":

```
db.students.find({subjects: {$all: ["Math", "Science"]}})
```

Querying Nested Documents

- ❑ MongoDB allows querying fields within nested documents using dot notation.

- ❑ Example:

- Find all students whose "address.city" is "New York":

```
db.students.find({"address.city": "New York"})
```


Sorting Query Results

❑ Sorting Syntax:

- `db.collection.find().sort({<field>: 1 or -1})`
- `1` for ascending, `-1` for descending.

❑ **Example:** Sort students by age in ascending order:

```
db.students.find().sort({age: 1})
```

Limiting Query Results

❑ Limiting Syntax:

```
db.collection.find().limit(<number>)
```

❑ Example: Retrieve only the first 5 students:

```
db.students.find().limit(5)
```

Skip and Limit Example

❑ Syntax:

```
db.collection.find().skip(<number>).limit(<number>)
```

❑ Example: Skip the first 10 students and return the next 5:

```
db.students.find().skip(10).limit(5)
```

Full Text Search

- ❑ MongoDB supports full-text search with indexes.
- ❑ **Step 1:** Create a full-text index on the `bio` field:

```
db.students.createIndex({bio: "text"})
```

- ❑ **Step 2:** Query the text index:

```
db.students.find({$text: {$search: "data science"}})
```

Aggregation Framework

❑ Definition: Aggregation is used to process data and return computed results.

❑ Example:

- Find average age

```
db.collection.aggregate([  
  {$group: {_id: null, avgAge: {$avg: "$age"}}}  
])
```

❑ Stages:

- \$match, \$group, \$project, \$sort

Aggregation with Queries

❑ You can combine queries with MongoDB's aggregation pipeline to perform complex data processing.

❑ **Example:**

Find students whose age is greater than 20 and then group them by course:

```
db.students.aggregate([
  {$match: {age: {$gt: 20}}},
  {$group: {_id: "$course", count: {$sum: 1}}}
])
```

Query by Date

❑ MongoDB allows querying by date fields.

❑ **Example:**

- Find students who joined after January 1st, 2022:

```
db.students.find({  
  joinDate: {$gt: new ISODate("2022-01-01T00:00:00Z")}  
})
```

Exercises

Exercise 1 - Basic CRUD

❑ **Objective:** Perform basic CRUD operations in MongoDB.

1. Steps:
2. Insert a document into a collection.
3. Query the document.
4. Update the document.
5. Delete the document.

❑ **Example:**

```
db.students.insertOne({name: "Alice", age: 21, course: "Data Science"})
db.students.find()
db.students.updateOne({name: "Alice"}, {$set: {age: 22}})
db.students.deleteOne({name: "Alice"})
```

Exercise 2 - Aggregation Pipeline

❑ Objective:

Use the aggregation framework to calculate statistics.

❑ Steps:

1. Insert multiple documents.
2. Use aggregation to calculate the average age.

❑ Example:

```
db.students.insertMany([
  {name: "John", age: 25},
  {name: "Jane", age: 22},
  {name: "Tom", age: 24}
])

db.students.aggregate([
  {$group: {_id: null, avgAge: {$avg: "$age"}}}]
])
```

Exercise 3 - Indexing

❑ Objective:

Create an index to optimize queries.

❑ Steps:

- Create an index on the `name` field.
- Measure the performance before and after indexing.

❑ Example:

```
db.students.createIndex({name: 1})
```

Exercise 4 - Querying

❑ Objective:

- Practice querying a MongoDB collection using various conditions.

❑ Tasks:

1. Insert multiple documents into a books collection with fields like title, author, year, and genre.
2. Query the collection to find:
 - All books published after 2010.
 - Books by a specific author.
 - Sort the results by year in descending order.

Conclusion

- ❑ MongoDB is a powerful, flexible NoSQL database suited for a wide variety of applications.
- ❑ It allows for easy scalability and high performance.
- ❑ Mastering basic CRUD operations, aggregation, indexing, and security will give you a solid foundation.